

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Brownlee, Jason. "A Gentle Introduction to Ensemble Learning Algorithms." *Machine Learning Mastery*, 19 April 2021, <https://machinelearningmastery.com/tour-of-ensemble-learning-algorithms/>.

- **Required pages:** Entire article
- **Estimated reading time:** 30 minutes

2. Gray, Chad. "Stock Prediction with ML: Walk-forward Modeling." *The Alpha Scientist*, 18 July 2018, https://alphascientist.com/walk_forward_model_building.html.

- **Required pages:** Entire article
- **Estimated reading time:** 30 minutes

Module 7: Lesson 1

Advanced topics in Deep Learning for Finance: Ensembles and Walk Forward



Outline

- ▶ Ensemble methods
- ▶ Walk forward methods

Deep learning models and ensembles

Deep neural networks are flexible models, but they are prone to problems that damage their generalization capacity (bias and variance):

- ▶ The predictions from deep neural networks are sensitive to initial conditions, both in terms of the initial random weights and in terms of the statistical noise in the training dataset.
- ▶ The stochastic nature of the learning algorithm means that slightly (or dramatically) different versions of the mapping function from inputs to outputs are learned out of each initialization and different performances on the training and holdout (test) datasets are obtained.

Ensemble learning represents a useful tool to overcome these problems and improve the performance of predictions:

- ▶ Ensembles consist of training multiple models instead of a single model and combining the predictions from these models to obtain a single overall prediction.
- ▶ The individual models are “weak learners” that, combined, lead to a stronger learner that performs better than the individual ones.

Ensemble methods

Ensemble learning involves training more than one network on the same dataset, then using each of the trained models to make a prediction before combining the predictions in some way to make a final prediction.

The most commonly used ensemble approach for neural networks is called model-averaging or “committee of networks.”

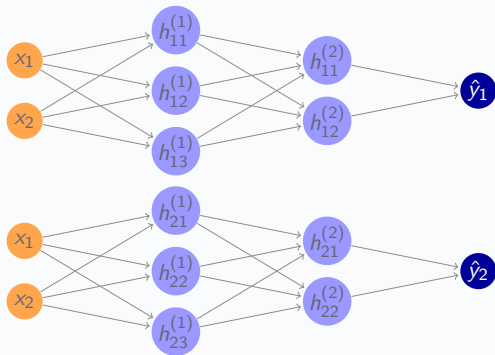
- ▶ A collection of networks with the same configuration and different initial random weights is trained on the same dataset. The actual overall predictions are calculated as the average of the predictions of each trained model.

The number of models in the ensemble is often kept small (three, five, or 10 trained models) both because of the computational expense in training models and because of the diminishing returns in performance from adding more ensemble members.

Model-averaging ensemble

Input

Output



Combined prediction



Advanced ensemble methods

The field of ensemble learning is well studied, and there are many improvements relative to model averaging. Techniques for ensemble learning can be classified according to which element is varied in the process.

Training data: Each model in the ensemble is trained with different data.

- ▶ K-fold cross-validation: k models are trained on k different subsets of the training data. The ensemble prediction comes from averaging the predictions from the $k - 1$ models where each observation belongs to the test data.
- ▶ Bootstrap aggregation (Bagging): resampling the training dataset with replacement, then training the model for several resampled datasets.

Models: The combined prediction is obtained from models with different configurations.

- ▶ The ensemble may learn heterogeneous mappings that may display a low correlation in their errors.

Combinations: The contribution of each model to the combined prediction is optimized.

- ▶ Stacking: Training an entirely new model to learn how to best combine the contributions from each member of the ensemble.
- ▶ Boosting: Sequentially train models that learn from the prediction errors of previous models.

Ensemble methods in financial applications

Ensemble methods are meant to generate a stable predictor that reduces the noise and randomness involved while training deep neural networks.

Some of the methods we outlined before involve slicing or resampling of the data (k -fold cross-validation and Bagging).

However, in most applications in finance, observations cannot be assumed to be independent and identically distributed.

- ▶ This means that slicing and resampling must be done with care.

Placing relatively close observations of the dataset in different sets will likely lead to some **leakage**.

- ▶ Information that is present in the training set (either in the inputs or the labels) is also present in the test set due to the serial correlation.
- ▶ Leakage is likely to lead to overfitting that contaminates the test samples and inflates performance in backtests.
- ▶ In Lesson 3, we will describe the “Combinatorial Purged Cross-Validation” as a tool to circumvent this issue.

Walk Forward

In most applications, our course of action in training deep learning models has been to first split the data by time, training on the first chunk of data and using the second for out-of-sample tests.

While this choice is not wrong by itself, we are not making proper use of the information.

- ▶ We exploit a low share of the information to evaluate the out-of-sample performance.
- ▶ We are biasing our tests based on the ability of the models to predict out of sample only in the most recent part of the time series.

We can opt, instead, for a **Walk Forward** method:

- ▶ We generate a sequence of out-of-sample evaluations by periodically re-training the model using a rolling, or expanding, time window.
- ▶ We can think of walk forward as an alternative resampling tool that preserves the sequential dimension of the data.

Walk Forward methods

The Walk forward method can be sketched as follows:

1. Select the length of the training and test samples, $\mathcal{T}_1$ and $\mathcal{T}_2$, respectively.
2. Train the model using a window of information that spans $\mathcal{T}_1$ periods starting with the initial date of the sample. Evaluate the model performance on a test sample that includes $\mathcal{T}_2$ periods and begins at some date after the end of the training sample.
3. Save the predictions of the trained model in the test sample as part of a result set.
4. Shift the training and test windows $\mathcal{T}_2$ periods.
5. Repeat steps 2 to 4, adding the new predictions to the result set until the test set reaches the end of the time series.

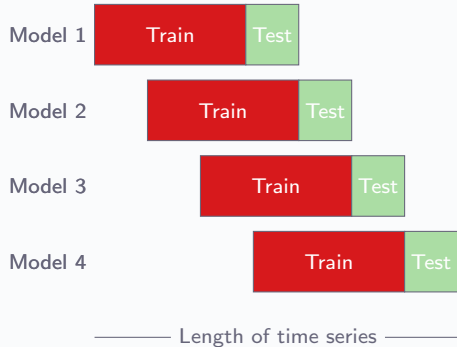
Financial features often incorporate serial correlation and the computation of the labels yields some overlap, e.g. when we want to predict stock returns over a 5-day window using daily observations.

- To avoid leakage, we should impose some window of excluded observations between the training and test samples (purging).

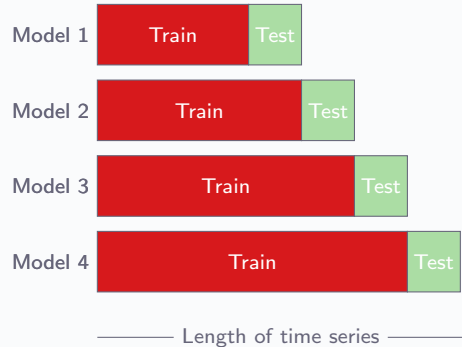
We can also opt for an “anchored” walk forward by using in the training samples all the information available since the initial date of the sample.

Walk Forward

Non-Anchored Walk Forward



Anchored Walk Forward



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Described the role of ensemble methods.
- ▶ Described the Walk Forward method.

⇒ **References for this lesson:**

Brownlee, Jason. "A Gentle Introduction to Ensemble Learning Algorithms." *Machine Learning Mastery*, April 2021.

Gray, Chad. "Stock Prediction with ML: Walk-forward Modeling." *The Alpha Scientist*, July 2018.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement the walk forward method in a prediction problem for Bitcoin.

See you there!

Module 7: Lesson 2

Advanced topics in Deep Learning for Finance: Application of Walk Forward



Outline

- ▶ The Walk Forward method in practice
- ▶ Application to hourly Bitcoin data

Walk Forward

In Lesson 1, we described the Walk Forward method as a tool to increase the robustness of our evaluations of trading strategies.

With a Walk Forward, we can perform backtests using more information than that of the last chunk of data from a standard train/test split, and we reduce the bias of our evaluations toward the most recent part of the data.

In Lesson 2, we take this method into practice, using a non-anchored Walk Forward (fixed-size rolling window).



Walk Forward: Application to Bitcoin

In our illustration of the Walk Forward, we exploit high-frequency Bitcoin price data.

The dataset covers hourly information, provided transactions have occurred, in the 2013–2022 period.

To ease the computational requirements, we only use information from the 2020–2022 period.

	date	volume	trades	Ret	year
43802	2020-01-01 00:00:00	70.761775	267	-0.001758	2020
43803	2020-01-01 01:00:00	224.921206	528	0.005408	2020
43804	2020-01-01 02:00:00	138.910528	336	0.003906	2020
43805	2020-01-01 03:00:00	97.687456	240	-0.002908	2020
43806	2020-01-01 04:00:00	25.305555	131	-0.001236	2020
...	...	...	...	...	...
67886	2022-09-30 19:00:00	290.023381	1947	-0.008941	2022
67887	2022-09-30 20:00:00	759.074836	1648	-0.003939	2022
67888	2022-09-30 21:00:00	564.894247	1245	-0.003090	2022
67889	2022-09-30 22:00:00	90.626135	1978	0.001379	2022
67890	2022-09-30 23:00:00	57.986112	2466	0.001965	2022

24089 rows × 5 columns

Walk Forward: Deep Learning Architecture

We employ an MLP architecture for a binary classification problem.

- ▶ The label to predict is the direction of the cumulative Bitcoin return over the next 4-hour interval.
- ▶ The input features are the lags of the hourly Bitcoin returns over the previous 15 days.

The MLP features three fully connected hidden layers with, respectively, 85, 55, and 30 hidden units.

- ▶ Each hidden layer features a ReLU activation function and a sigmoid function in the output layer.
- ▶ For regularization, we introduce Dropout after all the hidden layers and early stopping.

Walk Forward: Implementation

In the Walk Forward, we train the MLP separately for successive training samples of fixed size, using the procedure outlined in Lesson 1.

We set the size of the training window to $\mathcal{T}_1=5,000$ hours of data, while we set the test window to $\mathcal{T}_2=1,000$ hours.

The procedure involves nesting the usual lines of code that involve training a model in TensorFlow into a for loop of the form:

```
n_train = 5000
n_test = 1000

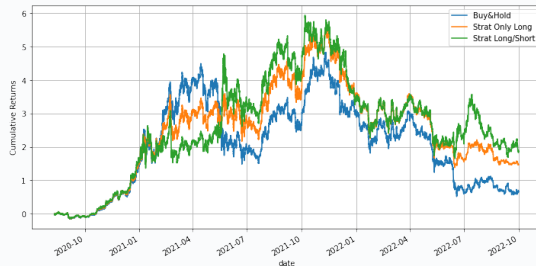
for i in range(0, len(y)-n_train, n_test):
    X_train, X_test = X[i:i+(n_train)], X[i+n_train:i+n_train+n_test]
    y_train, y_test = y[i:i+(n_train)], y[i+n_train:i+n_train+n_test]
```

We store in a pandas data frame the prediction results for each test observation.

Walk Forward: Results

Once we have trained the model for all the training windows, we can obtain backtest statistics for a larger set of data than the usual single training/testing split.

- We can obtain out-of-sample predictions for all the datasets, except for the initial training sample.



Disclaimer: The results above can vary due to the randomness of the training process.

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Revisited the Walk Forward method.
- ▶ Implemented the Walk Forward using high-frequency data.

⇒ **References for this lesson:**

Gray, Chad. "Stock Prediction with ML: Walk-forward Modeling." *The Alpha Scientist*, July 2018.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we will describe a more advanced method to obtain robust backtests: the Combinatorial Purged Cross-Validation.

See you there!

DEEP LEARNING FOR FINANCE

MODULE 7 | LESSON 2

THE WALK FORWARD METHOD: APPLICATION TO BITCOIN

Reading Time	50 minutes
Prior Knowledge	Neural Networks, Machine Learning
Keywords	Walk forward, Multi-Layer Perceptron, Backtesting

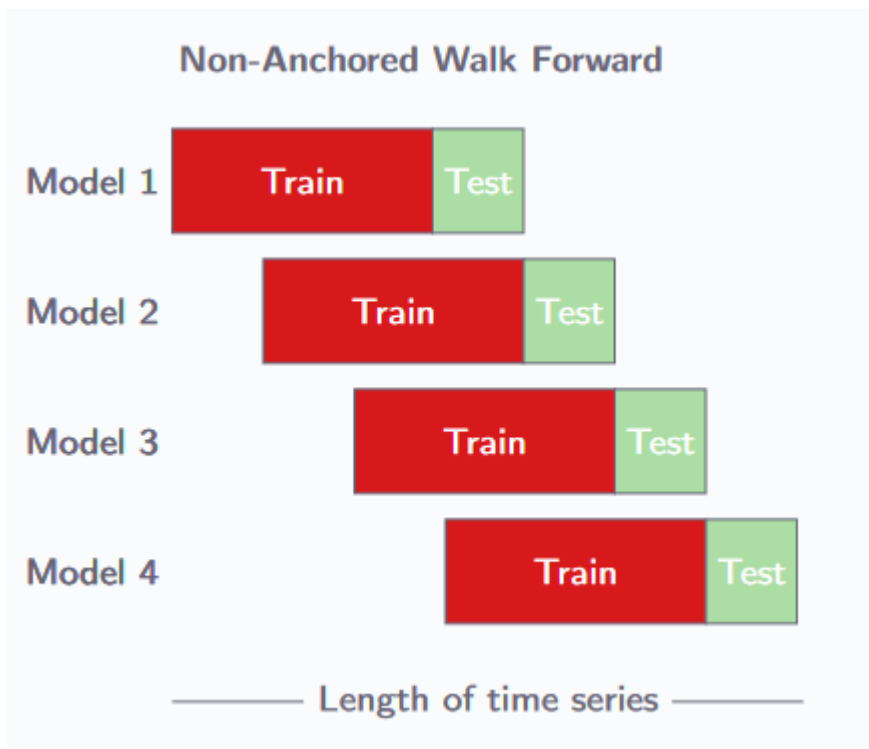
1. INTRODUCTION

In this notebook, we illustrate the implementation of the Walk Forward method as a backtest procedure for trading strategies based on deep learning models. We will exploit hourly data of Bitcoin prices and train a classification model to predict the direction of Bitcoin's return over 4-hour intervals.

In our previous applications of deep learning to trading strategies, we have always started by splitting the data by time into two parts, training on the first chunk of data and using the second for out-of-sample tests. This procedure is not wrong by itself but employs a very small part of the information to perform backtests. This means that we obtain estimates of the performance of our strategy from relatively few observations compared to the overall sample. Moreover, we are biasing our tests based on the ability of the models to predict out of sample only in the most recent part of the time series. For instance, the test sample may feature a booming period in security prices, and our model may perform well in those periods. However, this may obscure the fact that the model may perform terribly in downturn periods.

A simple way to overcome this--and have a general view of our trading strategies and how the underlying models perform over time--is to opt for a **Walk Forward** backtesting method. To implement a Walk Forward, we generate a sequence of out-of-sample evaluations by periodically re-training the model using a rolling, or expanding, time window.

The figure below illustrates the Walk Forward that we are going to implement below. This is a non-anchored method because we use a fixed rolling window of training data that we are going to shift sequentially so that the training set includes the test set in the previously trained model.



Before proceeding, we load all the required libraries and the data that we will exploit in this notebook. Among other information, the dataset covers hourly Bitcoin prices in 2013–2022. Notice that the earlier data does not include every-hour information due to the lack of transactions. For computational reasons, we will restrict the analysis to post-2020 data.

```
In [1]: # %%
import numpy as np
import pandas as pd
import tensorflow as tf

# %%
# =====
name = "Kraken_BTCUSD_60_3Q2022"
df = pd.read_csv(name + ".csv")

df["unix"] = pd.to_numeric(df["unix"])
df["date"] = pd.to_datetime(df["unix"], unit="s")
```

```
2025-05-26 12:39:11.666012: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 12:39:11.716667: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 12:39:13.064196: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [2]: df = df[["date", "close", "volume", "trades"]]
```

```
In [3]: df.head()
```

```
Out[3]:
```

	date	close	volume	trades
0	2013-10-06 21:00:00	122.00	0.1000	1
1	2013-10-07 20:00:00	123.61	0.1000	1
2	2013-10-08 02:00:00	123.90	1.9916	2
3	2013-10-08 05:00:00	124.18	2.0000	2
4	2013-10-09 09:00:00	123.84	2.8230	3

```
In [4]: df.tail()
```

```
Out[4]:
```

	date	close	volume	trades
67886	2022-09-30 19:00:00	19497.2	290.023381	1947
67887	2022-09-30 20:00:00	19420.4	759.074836	1648
67888	2022-09-30 21:00:00	19360.4	564.894247	1245
67889	2022-09-30 22:00:00	19387.1	90.626135	1978
67890	2022-09-30 23:00:00	19425.2	57.986112	2466

```
In [5]: df["Ret"] = df["close"].pct_change()
```

```
In [6]: df["year"] = df["date"].dt.year
```

```
In [7]: df.tail()
```

Out[7]:		date	close	volume	trades	Ret	year
	67886	2022-09-30 19:00:00	19497.2	290.0233 81	1947	-0.008941	2022
	67887	2022-09-30 20:00:00	19420.4	759.0748 36	1648	-0.003939	2022
	67888	2022-09-30 21:00:00	19360.4	564.8942 47	1245	-0.003090	2022
	67889	2022-09-30 22:00:00	19387.1	90.62613 5	1978	0.001379	2022
	67890	2022-09-30 23:00:00	19425.2	57.98611 2	2466	0.001965	2022

```
In [8]: del df["close"]
df = df.loc[(df["year"] >= 2020)]
df.head()
```

Out[8]:		date	volume	trades	Ret	year
	43802	2020-01-01 00:00:00	70.761775	267	-0.001758	2020
	43803	2020-01-01 01:00:00	224.921206	528	0.005408	2020
	43804	2020-01-01 02:00:00	138.910528	336	0.003906	2020
	43805	2020-01-01 03:00:00	97.687456	240	-0.002908	2020
	43806	2020-01-01 04:00:00	25.305555	131	-0.001236	2020

```
In [9]: df["Ret"] = df["Ret"].fillna(0)
df["volume"] = df["volume"].fillna(0)
df["trades"] = df["trades"].fillna(0)
df.head()
```

Out[9]:		date	volume	trades	Ret	year
	43802	2020-01-01 00:00:00	70.761775	267	-0.001758	2020
	43803	2020-01-01 01:00:00	224.921206	528	0.005408	2020
	43804	2020-01-01 02:00:00	138.910528	336	0.003906	2020
	43805	2020-01-01 03:00:00	97.687456	240	-0.002908	2020
	43806	2020-01-01 04:00:00	25.305555	131	-0.001236	2020

```
In [10]: df
```

```
Out[10]:
```

	date	volume	trades	Ret	year
43802	2020-01-01 00:00:00	70.761775	267	-0.001758	2020
43803	2020-01-01 01:00:00	224.921206	528	0.005408	2020
43804	2020-01-01 02:00:00	138.910528	336	0.003906	2020
43805	2020-01-01 03:00:00	97.687456	240	-0.002908	2020
43806	2020-01-01 04:00:00	25.305555	131	-0.001236	2020
...	...	...	...	...	...
67886	2022-09-30 19:00:00	290.023381	1947	-0.008941	2022
67887	2022-09-30 20:00:00	759.074836	1648	-0.003939	2022
67888	2022-09-30 21:00:00	564.894247	1245	-0.003090	2022
67889	2022-09-30 22:00:00	90.626135	1978	0.001379	2022
67890	2022-09-30 23:00:00	57.986112	2466	0.001965	2022

24089 rows × 5 columns

2. SETTING UP AND TRAINING THE FORECASTING MODEL

To illustrate the workings of Walk Forward, we will resort to a "plain vanilla" MLP architecture where the label to predict is the direction of the cumulative Bitcoin return over the next 4-hour interval. That is, we set up a binary classification problem where the positive class is a positive return and the negative class is a zero or negative return. The input features are the lags of the hourly Bitcoin returns over the previous 15 days, meaning that we have $24 \times 15 = 360$ features. Our final time series consists of more than 23,000 observations.

```
In [11]: df = df.reindex(
        columns=[
            "date",
            "Ret",
            "volume",
            "trades",
            "year",
            "month",
            "day",
            "week",
            "weekday",
            "hour",
        ]
    )
df = df[["date", "Ret"]]

n_lags = 24 * 15

for i in range(1, n_lags + 1):
    name = "Ret_" + str(i)
    df[name] = df["Ret"].shift(i)

df["Ret4_i"] = df["Ret"].rolling(4).apply(lambda x: 100 * (np.prod(1 + x /
100) - 1))
df["Ret4"] = df["Ret4_i"].shift(-4)
df["Output"] = df["Ret4"] > 0
df["Output"] = df["Output"].astype(int)
del df["Ret4"]
del df["Ret4_i"]

df = df.dropna()
```

```

/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)

```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```

/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)

```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```

/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)

```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```

/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/604952926.py:23: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Ret4_i"] = df["Ret"].rolling(4).apply(lambda x: 100 * (np.prod(1 + x /
100) - 1))
/tmp/ipykernel_44/604952926.py:24: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Ret4"] = df["Ret4_i"].shift(-4)
/tmp/ipykernel_44/604952926.py:25: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Output"] = df["Ret4"] > 0

```

```

In [12]: X, y = df.iloc[:, 2:-1], df.iloc[:, -1]
         print(X.shape, y.shape)

```

```
(23729, 360) (23729,)
```

The MLP's architecture includes three fully connected hidden layers with, respectively, 85, 55, and 30 hidden units. Each hidden layer features a ReLU activation function and a sigmoid function in the output layer. As regularization tools, we introduce Dropout after all the hidden layers and an early stopping criterion.

```

In [13]: hp_units = 85
         hp_units_2 = 55
         hp_units_3 = 30
         n_dropout = 0.2
         act_fun = "relu"

```

Below, we implement the full Walk Forward loop. The size of the training window is 5,000 observations, and the test window has 1,000 observations. Starting at the beginning of our dataset, we are going to train the model using 5,000 observations and test its out-of-sample performance for the following 1,000 observations. After that, we shift the training window 1,000 observations and repeat the process until we reach the end of the full sample. We store in a pandas data frame the prediction results for each test observation. All rights reserved WQU WorldQuant University QQQQ


```

In [14]: n_train = 5000
n_test = 1000

time_backtest = []
time_backtest = np.array(time_backtest)
time_backtest = time_backtest.astype("datetime64", copy=False)

pred_backtest = []
ret_backtest = []

for i in range(0, len(y) - n_train, n_test):
    X_train, X_test = X[i : i + (n_train)], X[i + n_train : i + n_train +
n_test]
    y_train, y_test = y[i : i + (n_train)], y[i + n_train : i + n_train +
n_test]
    test_time = df.iloc[i + (n_train) : i + n_train + n_test, 0:1].values
    Ret_vector = df.iloc[i + (n_train) : i + n_train + n_test, 1:2].values
    print(
        "ite ",
        i / n_test,
        " -- start test:",
        test_time[0],
        " -- end test",
        test_time[-1],
        " ----- ",
        X_train.shape,
        y_train.shape,
        X_test.shape,
        y_test.shape,
        test_time.shape,
        Ret_vector.shape,
    )

    tf.keras.backend.clear_session()
    tf.random.set_seed(1234)

    model = tf.keras.models.Sequential()
    model.add(tf.keras.layers.Dense(units=hp_units, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=hp_units_2, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=hp_units_3, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=1, activation="sigmoid"))

    hp_lr = 1e-5

    adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
    model.compile(optimizer=adam, loss="binary_crossentropy", metrics=
["accuracy"])

    es = tf.keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        mode="max",
        verbose=0,
        patience=20,
        restore_best_weights=True,
    )
    class_weight = {0: (np.mean(y_train) / 0.5), 1: 1}
    history = model.fit(

```

```
X_train,
y_train,
validation_split=0.15,
epochs=500,
batch_size=32,
verbose=0,
callbacks=[es],
class_weight=class_weight,
)

y_prob = model.predict(X_test, verbose=0)
y_pred = np.where(y_prob > 0.50, 1, 0)

time_backtest = np.append(time_backtest, test_time.flatten())
pred_backtest = np.append(pred_backtest, y_pred.flatten())
ret_backtest = np.append(ret_backtest, Ret_vector.flatten())

df_predictions = pd.DataFrame(
    {"Date": time_backtest, "Pred": pred_backtest, "Ret": ret_backtest}
)
```

```
ite 0.0 -- start test: ['2020-08-11T08:00:00.000000000'] -- end test
['2020-09-21T23:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 1.0 -- start test: ['2020-09-22T00:00:00.000000000'] -- end test
['2020-11-02T16:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 2.0 -- start test: ['2020-11-02T17:00:00.000000000'] -- end test
['2020-12-14T08:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 3.0 -- start test: ['2020-12-14T09:00:00.000000000'] -- end test
['2021-01-25T00:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 4.0 -- start test: ['2021-01-25T01:00:00.000000000'] -- end test
['2021-03-07T17:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 5.0 -- start test: ['2021-03-07T18:00:00.000000000'] -- end test
['2021-04-18T09:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 6.0 -- start test: ['2021-04-18T10:00:00.000000000'] -- end test
['2021-05-30T01:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 7.0 -- start test: ['2021-05-30T02:00:00.000000000'] -- end test
['2021-07-10T17:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 8.0 -- start test: ['2021-07-10T18:00:00.000000000'] -- end test
['2021-08-21T14:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 9.0 -- start test: ['2021-08-21T15:00:00.000000000'] -- end test
['2021-10-02T06:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 10.0 -- start test: ['2021-10-02T07:00:00.000000000'] -- end test
['2021-11-12T22:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 11.0 -- start test: ['2021-11-12T23:00:00.000000000'] -- end test
['2021-12-24T14:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 12.0 -- start test: ['2021-12-24T15:00:00.000000000'] -- end test
['2022-02-04T06:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 13.0 -- start test: ['2022-02-04T07:00:00.000000000'] -- end test
['2022-03-17T22:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 14.0 -- start test: ['2022-03-17T23:00:00.000000000'] -- end test
['2022-04-28T14:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 15.0 -- start test: ['2022-04-28T15:00:00.000000000'] -- end test
['2022-06-09T06:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 16.0 -- start test: ['2022-06-09T07:00:00.000000000'] -- end test
['2022-07-20T22:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 17.0 -- start test: ['2022-07-20T23:00:00.000000000'] -- end test
['2022-08-31T14:00:00.000000000'] ----- (5000, 360) (5000,) (1000, 360)
(1000,) (1000, 1) (1000, 1)
ite 18.0 -- start test: ['2022-08-31T15:00:00.000000000'] -- end test
['2022-09-30T23:00:00.000000000'] ----- (5000, 360) (5000,) (729, 360)
(729,) (729, 1) (729, 1)
```

```
In [15]: df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%YYYY-%mm-%dd")
df_res = df_predictions
df_res.tail()
```

```
Out[15]:
```

	Date	Pred	Ret
18724	2022-09-30 19:00:00	1.0	-0.008941
18725	2022-09-30 20:00:00	0.0	-0.003939
18726	2022-09-30 21:00:00	1.0	-0.003090
18727	2022-09-30 22:00:00	0.0	0.001379
18728	2022-09-30 23:00:00	0.0	0.001965

3. RESULTS

Using the predictions of the trained models over all the test samples, we can now perform our backtest and obtain estimates of the returns of the underlying trading strategy. We will compare the cumulative returns of three different strategies:

1. A buy-and-hold strategy that is long on Bitcoin as of the beginning of the sample period.
2. A Long-Short strategy that arises from the predictions of the MLP: The position is long one unit when the predicted probability of a positive return over 4 hours is greater than 0.5 and the position is short one unit otherwise.
3. A Long-only strategy that arises from the predictions of the MLP. The position is long one unit when the predicted probability of a positive return over 4 hours is greater than 0.5 and the position is zero otherwise.

Below, we report the cumulative returns and display the evolution of returns of these three strategies. Note that while we have fixed some seeds for the random number generators, there is some additional randomness involved and the results may vary each time you run the code.

```
In [16]: df_res["Positions"] = np.where(df_res["Pred"] > 0.5, 1, -1)
df_res["Positions_shift"] = df_res["Positions"].shift(1)
df_res["Strat_ret"] = df_res["Positions"].shift(1) * df_res["Ret"]
df_res["Positions_L"] = df_res["Positions"].shift(1)
df_res["Positions_L"][df_res["Positions_L"] == -1] = 0
df_res["Strat_ret_L"] = df_res["Positions_L"] * df_res["Ret"]
df_res["CumRet"] = df_res["Strat_ret"].expanding().apply(lambda x:
np.prod(1 + x) - 1)
df_res["CumRet_L"] = (
    df_res["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)
df_res["bhRet"] = df_res["Ret"].expanding().apply(lambda x: np.prod(1 + x)
- 1)

Final_Return_L = np.prod(1 + df_res["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df_res["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df_res["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_44/529454916.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_res["Positions_L"][df_res["Positions_L"] == -1] = 0
```

Strat Return Long Only = 143.26678717315295 %

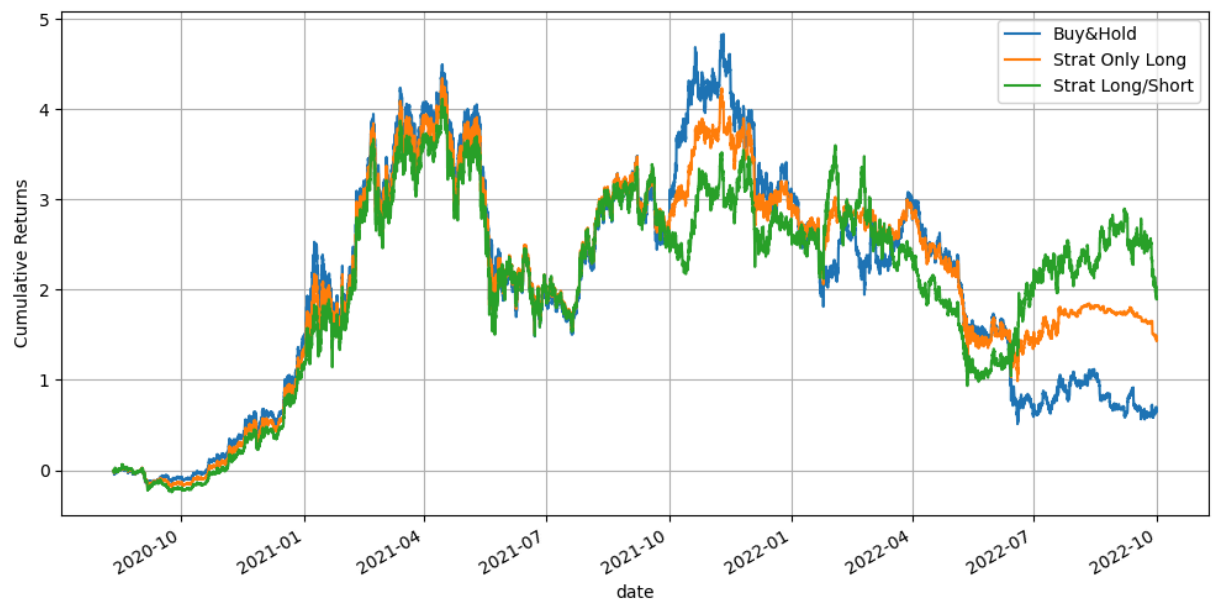
Strat Return = 190.02851215121103 %

Buy and Hold Return = 65.15499328334393 %

```
In [17]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df_res.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df_res.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df_res.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df_res["trade_bin"] = df_res["Positions"].diff().abs()
df_res["trade_bin"] = df_res["trade_bin"] / 2
df_res.describe()
```



Out[17]:

	Date	Pre d	Ret	Position s	Position s_shift	Strat_re t	Position s_L	Strat_re t_L	CumR et	CumR et_L	bhR et	trad e_bi n
count	18729	18729.00000	18729.00000	18729.00000	18728.00000	18728.00000	18728.00000	18728.00000	18728.00000	18728.00000	18729.00000	18728.00000
mean	2021-09-05 16:28:15.338779648	0.774895	0.00060	0.549789	0.549872	0.00090	0.774936	0.00075	2.054493	2.120305	2.053200	0.184910
min	2020-08-11 08:00:00	0.000000	-0.121059	-1.000000	-1.000000	-0.121059	0.000000	-0.121059	-0.241592	-0.185183	-0.154755	0.000000
25%	2021-02-22 12:00:00	1.000000	-0.003351	1.000000	1.000000	-0.003339	1.000000	-0.002189	1.508844	1.510123	0.821464	0.000000
50%	2021-09-05 19:00:00	1.000000	0.00090	1.000000	1.000000	0.000128	1.000000	0.000000	2.332326	2.215895	2.209577	0.000000
75%	2022-03-19 21:00:00	1.000000	0.003556	1.000000	1.000000	0.003580	1.000000	0.002448	2.829753	2.969356	3.033303	0.000000
max	2022-09-30 23:00:00	1.000000	0.124064	1.000000	1.000000	0.124064	1.000000	0.124064	4.110985	4.340209	4.832441	1.000000
std	NaN	0.417663	0.008154	0.835326	0.835271	0.008154	0.417636	0.007445	1.091337	1.155828	1.292641	0.388235

4. CONCLUSION

In this notebook, we have outlined and implemented the Walk Forward method to increase the robustness of our backtests for strategies based on Deep Learning methods.

In Lesson 3, we will describe a more advanced method to robustify our backtests: the Combinatorial Purged Cross-Validation.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Gort, Berend, and Bruce Yang. "The Combinatorial Purged Cross-Validation Method." *Towards AI*, 31 March 2022, <https://towardsai.net/p/l/the-combinatorial-purged-cross-validation-method>.

- **Required pages:** Entire article
- **Estimated reading time:** 40 minutes

1. López de Prado, Marcos. "Advances in Financial Machine Learning: Lecture 4/10." ORIE 5256, Cornell University. Class lecture. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3257420.

- **Required pages:** Slides 13–17
- **Estimated reading time:** 20 minutes

Module 7: Lesson 3

Advanced topics in Deep Learning for Finance: Combinatorial Purged Cross-Validation



Outline

- ▶ Pitfalls of Walk Forward and K-fold Cross-Validation
- ▶ Combinatorial Purged Cross-Validation

Pitfalls of Walk Forward and K-fold Cross-Validation

In Lessons 1 and 2, we described and implemented the **walk forward** method to test the robustness of a DL-based trading strategy.

While the walk forward method is not wrong by itself, it suffers from some weaknesses:

- ▶ The walk forward only tests over a single scenario (the historical path), meaning that each observation only has one forecast.
- ▶ The results can be biased by the specific behavior of the data in the training sample.

Instead, we may opt for **cross-validation**:

- ▶ E.g., training a model for the S&P500 in the mostly-booming period 2010–2019 and testing its performance in the 2007–2008 crisis period.
- ▶ However, there may be leakage of information from the test to the training sample that may generate misleading results.
- ▶ We only get one forecast per observation, and we only obtain an interpretation of how a strategy would have performed under hypothetical paths.

Combinatorial Purged Cross-Validation

The Combinatorial Purged Cross-Validation (CPCV) method addresses the issue that WF and CV only test a single path.

With CPCV, we can generate a prespecified number φ of backtesting paths by generating different combinations of training/testing sets.

- ▶ While it purges the observations to avoid leakage.

Consider a time series of T observations (labels and input features) partitioned into N groups without shuffling.

- ▶ Each group is of size $\lfloor T/N \rfloor$, except for the last group, which is of size $T - \lfloor T/N \rfloor(N - 1)$.

Let $k \leq N/2$ denote the number of groups of observations that we want to use as a testing set from training a model with the remaining $N - k$ groups.

- ▶ The total number of tested groups is $k \binom{N}{N - k}$.

The total number of “paths” that we can backtest, given the choices of N and k , is equal to:

$$\varphi[N, k] = \frac{k}{N} \binom{N}{N - k}$$

Combinatorial Purged Cross-Validation: Train-Test splits

Let's consider splitting a dataset into $N = 4$ groups, G1, G2, G3, G4, out of which we want to form combinations of train-test splits where each test set includes $k = 2$ groups.

For each train/test split S1,..., S6, the table below reports, with "x"s, which groups belong to the test set.

	S1	S2	S3	S4	S5	S6
G1	x	x	x			
G2	x			x	x	
G3		x		x		x
G4			x		x	x

Combinatorial Purged Cross-Validation: Backtest paths

With the choice of $N = 4$ and $k = 2$, we can form $\varphi[4, 3] = 3$ backtest paths from the combinatorial method by concatenating the predictions in the different test samples.

	S1	S2	S3	S4	S5	S6
G1	1	2	3			
G2	1			2	3	
G3		1		2		3
G4			1		2	3

With these combinations, we can obtain a “distribution” of backtest statistics (say Sharpe ratios), instead of having a single estimate for each statistic.

- In contrast, in a standard k -fold CV or Walk Forward, we would obtain a single backtest path.

CPCV: Purging

The procedure above shares similarities with the K-fold Cross-Validation method, where some test sets precede the training data.

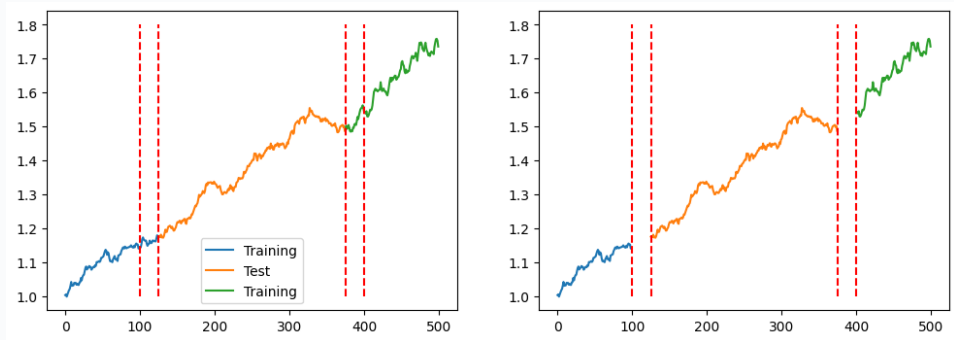
- ▶ This feature may lead to a leakage problem that can induce overfitting and that we must solve.
- ▶ The influence of leakage on the backtest statistics can be evaluated by taking $k \rightarrow T$.

Purging deals with those labels that overlap in their construction period.

- ▶ We must remove (purge) from the training set any observation whose label is based on the information that is also used in the test set.
- ▶ For example, consider the prediction problem for the 25-day return of an asset using daily data. To avoid leakage, we must purge from the training set the observations that include information 25 days before (after) the beginning (end) of the test sample.

By purging, we should not observe a monotonic improvement in performance out-of-sample as $k \rightarrow T$.

CPCV: Purging



CPCV: Embargoing

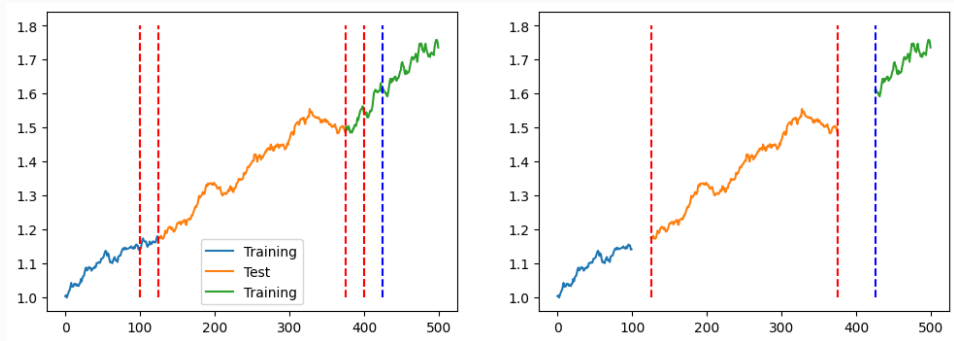
Purging may not prevent all leakage from the test to the training set due to the likely intrinsic serial correlation of the dataset:

- ▶ This problem only arises when the training set trails the test set.
- ▶ Information at the end of the test set may be serially correlated with that appearing in training information that goes after the test set.
- ▶ This correlation is a likely source of backtest overfitting.

We can **embargo** those training observations that are relatively close to the end of the test sample.

By embargoing, we should not observe a monotonic improvement in performance out of sample as $k \rightarrow T$.

CPCV: Purging and embargoing



CPCV: The algorithm

The full CPCV backtesting algorithm can be described as follows:

1. Partition the T observations of labels and input features into N groups without reshuffling. Groups $n = 1, \dots, N - 1$ are of size $\lfloor N/T \rfloor$ and group N is of size $T - \lfloor N/T \rfloor(N - 1)$.
2. Compute all the combinations of training/test splits. For each split, $N - k$ groups represent the training set, and k groups represent the test set.
3. For each train/test split, purge all the observations from the training sets whose labels span the period of the test sets. Apply an embargo to training observations that are preceded by the test set.
4. Train your model separately for all the combinations of training sets and compute predictions for their corresponding test sets.
5. Build the $\varphi[N, k]$ backtesting paths and obtain the distribution of backtest statistics of your choice.

For a detailed Python implementation, please check the references for this lesson.

Summary of Lesson 3

In Lesson 3, we have:

- Described the main ingredients of Combinatorial Purged Cross-Validation as a backtesting procedure.

⇒ **References for this lesson:**

López de Prado, M. "Advances in Financial Machine Learning: Lecture 4/10 (seminar slides)." *SSRN*. 29 September 2018.

Berend. "The Combinatorial Purged Cross-Validation method." *Medium*, 31 March 2022.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will describe how to introduce custom loss functions to train deep learning models in TensorFlow-Keras.

See you there!

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Keras. "Losses." <https://keras.io/api/losses/>.

- **Required pages:** Entire article
- **Estimated reading time:** 15 minutes

2. López de Prado, Marcos. "Advances in Financial Machine Learning: Lecture 4/10." ORIE 5256, Cornell University. Class lecture. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3257420.

- **Required pages:** Slides 1–16
- **Estimated reading time:** 25 minutes

3. Anis, Ahmad. "How To Build Custom Loss Functions in Keras for Any Use Case." *cnvrg.io*, <https://cnvrg.io/keras-custom-loss-functions/>.

- **Required pages:** Entire article
- **Estimated reading time:** 20 minutes

Module 7: Lesson 4

Advanced topics in Deep Learning for Finance: Custom Losses and Backtest Metrics



Outline

- ▶ Custom Loss functions and Backtest metrics
- ▶ Application to Bitcoin

Building custom losses in TensorFlow-Keras

Most of the time we use our models to develop investment strategies, whose performance metrics are defined in terms of risk-adjusted returns and other variables.

Typical loss functions that measure the predictive power of the models may not be appropriate targets or may not provide a general view of the quality of a predictive model.

In this lesson, we provide guidelines on how to build custom losses that allow for keeping track of the trading performance of the model while training and backtesting.

- The aim is that, after reading and analyzing the associated notebook, you can build loss functions and metrics that are tailored to the applications that you may encounter in the future.

General approach

Any function defined with the signature

```
loss_fn(y_true, y_pred)
```

can be passed to `compile()` as a loss function.

Both inputs are tensors with dimension (batch_size, number of outputs).

For instance, we can define a custom MSE loss function for a regression problem as follows and then pass it to our model.

```
def custom_mse(y_true, y_pred):  
    loss0 = tf.square(y_pred - y_true)  
    loss1 = tf.reduce_sum(loss0, axis=-1)  
    return loss1  
model.compile(loss=custom_mse, optimizer='adam')
```

At each optimization step, `fit()` will compute the average of “loss” across observations in the current batch.

Adding parameters

If our loss functions include some parameter, we need to define a **wrapper function** that includes the parameter of interest as an argument and that also nests the loss function of interest.

For instance, suppose that our model generates several outputs, and we want to give particular relevance to some of those outputs over others with a vector of weights.

```
def wrapper(w) # w is the vector of weights
    def custom_mse(y_true, y_pred):
        loss = tf.square(y_pred - y_true)
        loss_w = tf.math.multiply(loss, w) # apply weights
        loss_w_sum = tf.reduce_sum(loss_w, axis=-1)
        return loss_w_sum
    return custom_mse
```

We can pass the wrapper function to the compile method, after specifying the vector of weights.

```
weights = [0.1, ..., 0.025]
model.compile(loss=wrapper(weights), optimizer='adam')
```

Custom backtest statistics (1): Accuracy

We can apply the same logic to define backtest metrics that evaluate the performance of the trading strategy that arises from the trained model.

The backtest metrics that we define next are based on and tailored to the application of Lesson 4 focused on the prediction of Bitcoin returns.

For instance, we can compute an accuracy metric that measures the frequency with which the sign of the return prediction is the same as the actual prediction:

```
def custom_metric_acc(y_true , y_pred):  
    trues = tf.cast(tf.math.multiply(y_true , y_pred) > 0, tf.int32)  
    trues_sum = tf.reduce_sum(trues)  
    acc = trues / tf.size(y_pred)  
    return acc
```

Custom backtest statistics (2): R^2

We can also keep track of the model's R^2 over our backtest.

```
def custom_metric_r2(benchmark):  
    def r2(y_true, y_pred):  
        ss_pred = tf.math.reduce_sum(tf.square(y_true - y_pred))  
        ss_b = tf.math.reduce_sum(tf.square(y_true - benchmark))  
        return 1 - ss_pred / ss_b  
    return r2
```

We would obtain the typical out-of-sample R^2 if we set “benchmark” equal to the average return in the training sample and compute the function above in the test set.

Custom backtest statistics (3): Sharpe Ratios

Instead of focusing on forecasting power, we can directly focus on risk-weighted return performance of the strategy, through the Sharpe ratio.

```
def custom_metric_sr(b):  
    def sharpe_ratio(y_true, y_pred):  
        strat_ret = strategy_rets(y_true, y_pred, b)  
        ret_mean = tf.reduce_mean(strat_ret)  
        ret_std = tf.math.reduce_std(strat_ret)  
        sr = tf.math.divide(ret_mean, ret_std)  
        sr = tf.math.sqrt(252) * sr  
        return sr  
    return sharpe_ratio
```

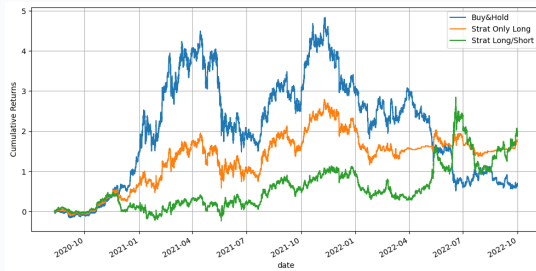
“strategy_rets” must be a function that determines (i) the position taken on the security given the predicted value and, possibly, some parameter(s) “b” and (ii) the return obtained given the position and the actual return.

Application

In the Jupyter Notebook for Lesson 4, we implement the construction of custom measures revisiting the Bitcoin return prediction problem that we already explored in Lesson 2.

We implement a Walk Forward procedure that keeps track of the backtesting metrics defined above.

- We also build other interesting metrics such as the Maximum Drawdown and Time under Water.



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built custom losses and backtest metrics in TensorFlow-Keras.
- ▶ Applied the computation of custom metrics in the evaluation of DL trading algorithms.

⇒ **References for this lesson:**

Anis, A. "How To Build Custom Loss Functions in Keras for Any Use Case." *convrg.io*.

López de Prado, M. "Advances in Financial Machine Learning: Lecture 6/10 (seminar slides)." *SSRN*. 6 October 2018.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 4.

This finalizes Module 7 and our journey in Deep Learning for Finance. Congrats!

DEEP LEARNING FOR FINANCE

MODULE 7 | LESSON 4

CUSTOM LOSS FUNCTIONS AND BACKTEST STATISTICS IN TENSORFLOW-KERAS

Reading Time	50 minutes
Prior Knowledge	Neural Networks, Machine Learning
Keywords	Custom Loss, Walk forward, Multi-Layer Perceptron, Backtesting

1. INTRODUCTION

In this notebook, we illustrate how to build custom loss functions when training deep learning models in TensorFlow-Keras. So far, we have trained our models using loss functions that measure the ability of the models to predict the variation in the training (or validation) set. However, this might not be the appropriate strategy in financial settings. Most of the time, we use our models to develop investment strategies whose performance metrics are defined in terms of risk-adjusted returns and other variables. The fact that a model yields a greater out-of-sample R^2 than other models does not necessarily mean that it will lead to better strategies in terms of standard metrics such as the Sharpe ratio.

This notebook provides some guidelines on how to build custom losses that allow for keeping track of the trading performance of the model while training and backtesting a model in a Walk Forward method, as introduced in lesson 2. The aim is that, after reading and analyzing this notebook, you can build loss functions that are tailored to the applications that you may encounter in the future.

Before proceeding, we load all the required libraries and the data that we will exploit in this notebook. We use the same information that we exploited in lesson 2's notebook.

```
In [1]: # %%
import numpy as np
import pandas as pd
import tensorflow as tf

# %%
# =====
name = "Kraken_BTCUSD_60_3Q2022"

df = pd.read_csv(name + ".csv")

df["unix"] = pd.to_numeric(df["unix"])
df["date"] = pd.to_datetime(df["unix"], unit="s")
```

2025-05-26 12:50:46.550595: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-26 12:50:46.602301: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-26 12:50:47.975789: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

```
In [2]: df = df[["date", "close", "volume", "trades"]]
```

```
In [3]: df["Ret"] = df["close"].pct_change()
df["year"] = df["date"].dt.year
```

```
In [4]: del df["close"]
df = df.loc[(df["year"] >= 2022)]
df["Ret"] = df["Ret"].fillna(0)
df["volume"] = df["volume"].fillna(0)
df["trades"] = df["trades"].fillna(0)
```

```

In [5]: df = df.reindex(
        columns=[
            "date",
            "Ret",
            "volume",
            "trades",
            "year",
            "month",
            "day",
            "week",
            "weekday",
            "hour",
        ]
    )
df = df[["date", "Ret"]]

n_lags = 24 * 15

for i in range(1, n_lags + 1):
    name = "Ret_" + str(i)
    df[name] = df["Ret"].shift(i)

df["Ret4_i"] = df["Ret"].rolling(4).apply(lambda x: 100 * (np.prod(1 + x / 100) - 1))
df["Ret4"] = df["Ret4_i"].shift(-4)
df["Output"] = df["Ret4"]

del df["Ret4"]
del df["Ret4_i"]

df = df.dropna()

```

```

/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)

```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```

/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)

```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
```

```

/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:21: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df[name] = df["Ret"].shift(i)
/tmp/ipykernel_44/3839557224.py:23: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Ret4_i"] = df["Ret"].rolling(4).apply(lambda x: 100 * (np.prod(1 + x /
100) - 1))
/tmp/ipykernel_44/3839557224.py:24: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Ret4"] = df["Ret4_i"].shift(-4)
/tmp/ipykernel_44/3839557224.py:25: PerformanceWarning: DataFrame is highly
fragmented. This is usually the result of calling `frame.insert` many
times, which has poor performance. Consider joining all columns at once
using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
    df["Output"] = df["Ret4"]

```

```

In [6]: X, y = df.iloc[:, 2:-1], df.iloc[:, -1]
        print(X.shape, y.shape)

```

```

(6188, 360) (6188,)

```

2. DEFINING CUSTOM LOSSES AND BACKTEST STATISTICS IN TENSORFLOW-KERAS

While the TensorFlow-Keras libraries contain many useful in-built loss functions for general machine learning projects, sometimes, we need to implement some modifications or implement a new loss function tailored to our particular application. Let's learn how to do that. Generally, any function (or *callable*) with the signature

```
loss_fn(y_true, y_pred)
```

and that generates a loss can be passed to `compile()` as a loss function. Both inputs are tensors with dimension `(batch_size, number of outputs)`.

For instance, we can define a custom MSE loss function as follows

```
def custom_mse(y_true, y_pred):
    # calculating squared difference between target and
    predicted values
    loss0 = tf.square(y_pred - y_true) # dimension
    (batch_size, number of outputs)
    loss1 = tf.reduce_sum(loss0, axis=-1) # dimension
    (batch_size, )
    return loss1
```

and then pass it to our model

```
model.compile(loss=custom_mse, optimizer='adam')
```

In the definition of `custom_mse`, the line `loss = tf.reduce_sum(y_pred - y_true, axis=-1)` takes into account that the model may generate several outputs. Such line can be replaced by `loss = tf.reduce_mean(y_pred - y_true, axis=-1)` without affecting the optimization process.

Below, we are going to train a regression model, with an MLP, to predict the returns of Bitcoin. Despite training a regression model, we still may be interested in tracking the ability of the model to predict the sign of returns, as in a classification model. For this, we can define a custom accuracy metric from the predictions of the model and the actual realizations of returns:

```
In [7]: def custom_metric_acc(y_true, y_pred):
    acc = tf.reduce_sum(
        tf.cast(tf.math.multiply(y_true, y_pred) > 0, tf.int32)
    ) / tf.size(y_pred)
    return acc
```

Sometimes, our loss functions include some parameter that is independent from the trainable parameters of the model. To allow this, we need to define a *wrapper function* that includes the parameter of interest as an argument and that also nests the loss function of interest.

For instance, suppose that our model generates several outputs (that is, predictions), and we want to give some particular relevance to some of those outputs over others with a predetermined vector of weights. We can define the wrapper function that includes as argument the vector of weights, as follows:

```
def wrapper(w)
    def custom_mse(y_true, y_pred):
        # calculating squared difference between target and
        predicted values
        loss = tf.square(y_pred - y_true) # dimension
        (batch_size, number of outputs)
        loss_w = tf.math.multiply(loss, w)
        loss_w_sum = tf.reduce_sum(loss_w, axis=-1) # dimension
        (batch_size, )
        return loss_w_sum
    return custom_mse
```

We can pass the wrapper function to the compile method, after specifying the vector of weights.

```
weights = [0.1, ..., 0.025]

model.compile(loss=wrapper(weights), optimizer='adam')
```

Instead of optimizing our model using the custom loss, we can also add it as an additional evaluation metric as follows:

```
model.compile(loss='mse', metrics = wrapper(weights),
optimizer='adam')
```

This option also allows for using the custom metric in an early stopping callback. After training, we can evaluate the performance in the test sample using the `evaluate()` method:

```
model.evaluate(X_test, y_test, batch_size = n_test)
```

where we set `batch_size` equal to the size of the test sample to obtain a proper evaluation of the loss function over the whole test sample. This will give us the value of both the in-built MSE loss and the custom MSE loss in the test sample.

Keras offers a variety of in-built loss functions. It is of relevance that you get familiar with the computation of these losses as described in the [documentation](#) to understand potential tweaks that you may have to implement in your own applications. In particular, you should review the computation of the following losses:

1. Regression loss functions
2. Mean Squared Error
3. Mean Absolute Error
4. Mean Squared Logarithmic Error
5. Mean Absolute Percentage Error
6. Huber loss
7. Poisson loss (for count data)
8. Classification loss functions
9. Binary Cross-Entropy loss
10. Hinge loss

The block of code below computes the R^2 of a regression model using a prediction `benchmark` against which the model is compared. For instance, the out-of-sample R^2 usually exploits the sample mean returns in the training test as the `benchmark`.

```
In [8]: def custom_metric_r2(benchmark):  
        def r2(y_true, y_pred):  
            return 1 - tf.math.reduce_sum(tf.square(y_true - y_pred)) /  
            tf.math.reduce_sum(  
                tf.square(y_true - benchmark)  
            )  
        return r2
```

Beyond metrics that estimate the predictive power of a model, we can build the metrics obtained from the trading strategies that are a byproduct of the trained models. Below, we define a Sharpe ratio function computed from a long-short strategy that is based on the predictions of the model, the actual returns in the data, and a symmetric prediction barrier. The prediction barrier sets the level of predicted returns that should trigger a long, short, or neutral position.

```
In [9]: def positions(y_pred, pred_barrier):
        """
        This function determines the long-short position on an asset based
        on the signal (predicted returns) y_pred from a model.
        pred_barrier is the threshold prediction that determines the long/short
        position
        """
        positions_long = tf.where(y_pred > pred_barrier, 1.0, 0.0)
        positions_short = tf.where(y_pred < -pred_barrier, -1.0, 0.0)
        return tf.add(positions_long, positions_short)

def strategy_rets(y_true, y_pred, pred_barrier):
    """
    This function computes the time series of returns given the
    predictions, actual returns
    and threshold prediction
    """
    return tf.math.multiply(positions(y_pred, pred_barrier)[:], y_true)

def custom_metric_sr(pred_barrier):
    """
    This function computes the annualized Sharpe Ratio from hourly return
    data
    """

    def sharpe_ratio(y_true, y_pred):
        strat_ret = strategy_rets(y_true, y_pred, pred_barrier)
        sr = tf.math.sqrt(24.0 * 365.0) * tf.math.divide(
            tf.reduce_mean(strat_ret), tf.math.reduce_std(strat_ret)
        )
        return sr

    return sharpe_ratio
```

One problem with the long-short strategy defined above is that the gradient of the loss function with respect to `y_pred` is zero everywhere (and not differentiable at `pred_barrier`) and TensorFlow will have problems in efficiently training the model if such function is used as the model's loss. An alternative is to define a strategy whose position in the asset is, say, linear in the prediction. This can be interpreted as defining a managed strategy where we put a fraction w of our wealth in an asset, and w is an increasing function of `y_pred`. We do this below by defining $w = y_pred / scale$, where *scale* can be a parameter that governs the sensitivity of the strategy to changes in the predicted return. For instance, *scale* can measure risk aversion and be itself a function of a risk metric, such as the variance of returns `y_true` over the training sample.

```
In [10]: def positions_managed(y_pred, scale):
    return tf.math.divide(y_pred, scale)

def strategy_rets_managed(y_true, y_pred, scale):
    return tf.math.multiply(positions_managed(y_pred, scale)[: , 0],
y_true)

def custom_metric_sr_managed(scale):
    def sharpe_ratio(y_true, y_pred):
        strat_ret = strategy_rets_managed(y_true, y_pred, scale)
        sr = tf.math.sqrt(24.0 * 365.0) * tf.math.divide(
            tf.reduce_mean(strat_ret), tf.math.reduce_std(strat_ret)
        )
        return sr

    return sharpe_ratio
```

Other useful backtesting metrics are the [Maximum Drawdown](#) (the maximum loss suffered by an investment between two consecutive High-watermarks (Running maxima). The Maximum Drawdown is then the maximum among those losses. The Maximum Time under Water determines the longest period of time between two High-watermarks. These measures are informative about a strategy's downside risk and the length and depth of losses before the strategy runs a profit.

(Note: The functions below cannot be passed to `compile()` as loss functions because they make use of pandas, which cannot be interpreted by TensorFlow).

```
In [11]: def custom_metric_DD(pred_barrier):
    def maxdrawdown(y_true, y_pred):
        # compute series of drawdowns
        strat_ret = strategy_rets_managed(y_true, y_pred, pred_barrier)
        cum_ret = tf.math.cumprod(1 + strat_ret).numpy()
        df0 = pd.DataFrame(cum_ret, columns=["cum_ret"])
        df0["HWM"] = df0["cum_ret"].expanding().max()
        df1 = df0.groupby("HWM").min().reset_index()
        df1.columns = ["HWM", "min"]
        df1.index = df0["HWM"].drop_duplicates(keep="first").index
        df1 = df1[df1["HWM"] > df1["min"]]
        dd = 1 - df1["min"] / df1["HWM"]
        return np.max(dd)

    return maxdrawdown
```

```
In [12]: def custom_metric_TuW(pred_barrier):
def time_under_water(y_true, y_pred):
    # compute the time under water
    strat_ret = strategy_rets_managed(y_true, y_pred, pred_barrier)
    cum_ret = tf.math.cumprod(1 + strat_ret).numpy()
    df0 = pd.DataFrame(cum_ret, columns=["cum_ret"])
    df0["HWM"] = df0["cum_ret"].expanding().max()
    df1 = df0.groupby("HWM").min().reset_index()
    df1.columns = ["HWM", "min"]
    df1.index = df0["HWM"].drop_duplicates(keep="first").index
    df1 = df1[df1["HWM"] > df1["min"]]
    tuw = df1.index[1:] - df1.index[:-1]
    tuw = pd.Series(tuw, index=df1.index[:-1])
    return np.max(tuw)

return time_under_water
```

3. SETTING UP AND TRAINING THE FORECASTING MODEL

Below, we set up the forecasting model and walk forward. Different from the notebook in lesson 2, we set up the model as a regression problem and also keep track of some performance metrics of the resulting strategies over each test sample.

Notice that the backtesting metrics defined above will return tensors with dimension `batch_size` where all elements will be the same. Then, it is important to set the batch size equal to the size of the test sample in the call to `model.evaluate()`. Otherwise, we would not get the correct metrics.

```

In [13]: n_train = 5000
n_test = 1000

time_backtest = []
time_backtest = np.array(time_backtest)
time_backtest = time_backtest.astype("datetime64", copy=False)

pred_backtest = []
ret_backtest = []

for i in range(0, len(y) - n_train, n_test):
    X_train, X_test = X[i : i + (n_train)], X[i + n_train : i + n_train +
n_test]
    y_train, y_test = y[i : i + (n_train)], y[i + n_train : i + n_train +
n_test]
    test_time = df.iloc[i + (n_train) : i + n_train + n_test, 0:1].values
    Ret_vector = df.iloc[i + (n_train) : i + n_train + n_test, 1:2].values

    y_benchmark = np.mean(y_train)

    tf.keras.backend.clear_session()
    tf.random.set_seed(1234)

    act_fun = "relu"
    hp_units = 85
    hp_units_2 = 55
    hp_units_3 = 30
    n_dropout = 0.2

    model = tf.keras.models.Sequential()
    model.add(tf.keras.layers.Dense(units=hp_units, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=hp_units_2, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=hp_units_3, activation=act_fun))
    model.add(tf.keras.layers.Dropout(n_dropout))
    model.add(tf.keras.layers.Dense(units=1))

    hp_lr = 1e-5

    adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)

    val_split = 0.15

    model.compile(
        optimizer=adam,
        loss="mse",
        metrics=[
            custom_metric_sr(0.0),
            custom_metric_r2(y_benchmark),
            custom_metric_acc,
        ],
    )

    es = tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        mode="min",
        verbose=0,
        patience=50,
        restore_best_weights=True,

```

```

)

history = model.fit(
    X_train,
    y_train,
    validation_split=val_split,
    epochs=500,
    batch_size=32,
    verbose=0,
    callbacks=[es],
)

# Performance out of sample
score = model.evaluate(X_test, y_test, batch_size=n_test, verbose=0)
y_pred = model.predict(X_test, verbose=0)
drawdown = custom_metric_DD(0.0)
timeuwater = custom_metric_TuW(0.0)
print(
    "ite ",
    i / n_test,
    " -- MSE: ",
    np.round(score[0], 4),
    " -- Sharpe Ratio: ",
    np.round(score[1], 4),
    " -- R200S: ",
    np.round(score[2], 4),
    " -- Acc.: ",
    np.round(score[3], 4),
)

time_backtest = np.append(time_backtest, test_time.flatten())
pred_backtest = np.append(pred_backtest, y_pred.flatten())
ret_backtest = np.append(ret_backtest, Ret_vector.flatten())

df_predictions = pd.DataFrame(
    {"Date": time_backtest, "Pred": pred_backtest, "Ret": ret_backtest}
)

```

```

ite 0.0 -- MSE: 0.0002 -- Sharpe Ratio: 4.8077 -- R200S: -0.0012 --
Acc.: 0.506
ite 1.0 -- MSE: 0.0002 -- Sharpe Ratio: -0.8226 -- R200S: -0.0117 --
Acc.: 0.4096

```

4. RESULTS OVER FULL BACKTEST PERIOD

Below, we plot the evolution of the strategy's return over the full backtesting period. As usual, note that while we have fixed some seeds for the random number generators, there is some additional randomness involved, and the results may vary each time you run the code.

```

In [14]: df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%YYYY-
%mm-%dd")
df_res = df_predictions
df_res.tail()

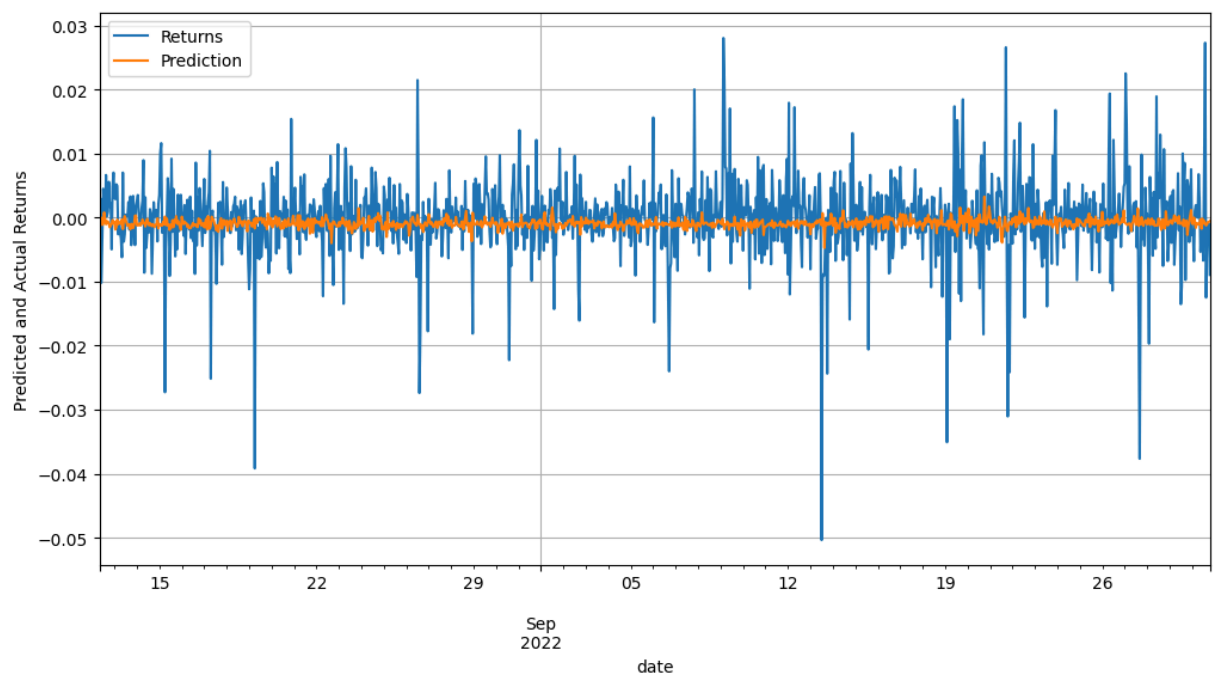
```

Out[14]:

	Date	Pred	Ret
1183	2022-09-30 15:00:00	-0.001219	-0.012480
1184	2022-09-30 16:00:00	-0.001036	-0.001959
1185	2022-09-30 17:00:00	-0.000684	-0.001040
1186	2022-09-30 18:00:00	-0.000682	-0.000919
1187	2022-09-30 19:00:00	-0.000545	-0.008941

In [15]: `import matplotlib.pyplot as plt`

```
fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df_res.plot(x="Date", y="Ret", label="Returns", ax=ax)
df_res.plot(x="Date", y="Pred", label="Prediction", ax=ax)
plt.xlabel("date")
plt.ylabel("Predicted and Actual Returns")
plt.grid()
plt.show()
```



```

In [16]: df_res["Positions"] = np.where(df_res["Pred"] > 0.0, 1, -1)
df_res["Positions_shift"] = df_res["Positions"].shift(1)
df_res["Strat_ret"] = df_res["Positions"].shift(1) * df_res["Ret"]
df_res["Positions_L"] = df_res["Positions"].shift(1)
df_res["Positions_L"][df_res["Positions_L"] == -1] = 0
df_res["Strat_ret_L"] = df_res["Positions_L"] * df_res["Ret"]

Final_Return_L = np.prod(1 + df_res["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df_res["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df_res["Ret"]) - 1

print(
    "Strat Return Long Only =",
    np.round(Final_Return_L * 100, 2),
    "% ,",
    "Sharpe ratio =",
    np.round(
        np.sqrt(24.0 * 365.0)
        * np.mean(df_res["Strat_ret_L"])
        / np.std(df_res["Strat_ret_L"]),
        2,
    ),
)
print(
    "Strat Return =",
    np.round(Final_Return * 100, 2),
    "% ,",
    "Sharpe ratio =",
    np.round(
        np.sqrt(24.0 * 365.0)
        * np.mean(df_res["Strat_ret"])
        / np.std(df_res["Strat_ret"]),
        2,
    ),
)
print(
    "Buy and Hold Return =",
    np.round(Buy_Return * 100, 2),
    "% ,",
    "Sharpe ratio =",
    np.round(np.sqrt(24.0 * 365.0) * np.mean(df_res["Ret"]) /
    np.std(df_res["Ret"]), 2),
)

```

Strat Return Long Only = -5.6 % , Sharpe ratio = -2.67

Strat Return = 4.82 % , Sharpe ratio = 0.89

Buy and Hold Return = -18.67 % , Sharpe ratio = -2.34

/tmp/ipykernel_44/2666742369.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

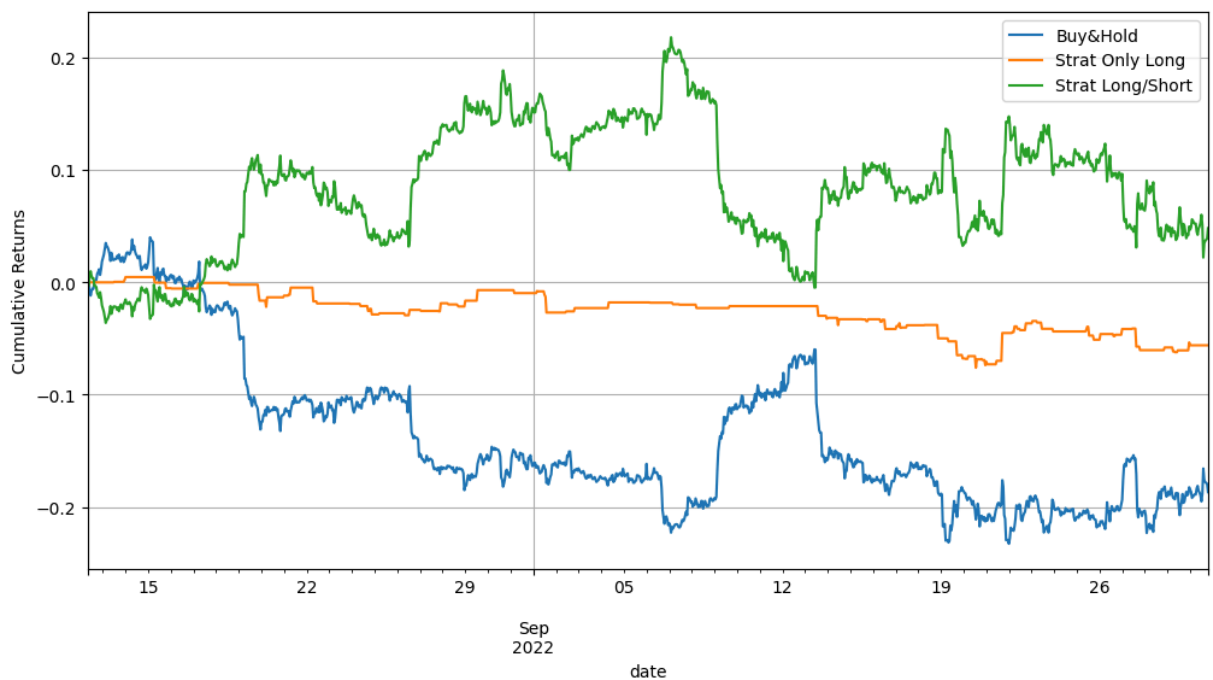
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df_res["Positions_L"][df_res["Positions_L"] == -1] = 0

```
In [17]: import matplotlib.pyplot as plt
```

```
df_res["CumRet"] = df_res["Strat_ret"].expanding().apply(lambda x:
np.prod(1 + x) - 1)
df_res["CumRet_L"] = (
    df_res["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)
df_res["bhRet"] = df_res["Ret"].expanding().apply(lambda x: np.prod(1 + x)
- 1)

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df_res.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df_res.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df_res.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df_res["trade_bin"] = df_res["Positions"].diff().abs()
df_res["trade_bin"] = df_res["trade_bin"] / 2
df_res.describe()
```



Out[17]:

	Date	Pre d	Ret	Position s	Position s_shift	Strat_re t	Position s_L	Strat_re t_L	Cum mRet	Cum mRet_L	bhRet	trade_bin
count	1188	1188.00000	1188.00000	1188.00000	1187.00000	1187.00000	1187.00000	1187.00000	1187.00000	1187.00000	1188.00000	1187.00000
mean	2022-09-06 01:30:00	-0.00879	-0.00155	-0.81698	-0.816344	0.000059	0.091828	-0.00047	0.082736	-0.025930	-0.137649	0.155013
min	2022-08-12 08:00:00	-0.004735	-0.050365	-1.00000	-1.00000	-0.028047	0.00000	-0.016800	-0.036132	-0.076005	-0.232680	0.00000
25%	2022-08-24 16:45:00	-0.001235	-0.002660	-1.00000	-1.00000	-0.002706	0.00000	-0.00000	0.043331	-0.038173	-0.187442	0.00000
50%	2022-09-06 01:30:00	-0.00928	0.00005	-1.00000	-1.00000	-0.000096	0.00000	0.00000	0.084821	-0.021515	-0.162404	0.00000
75%	2022-09-18 10:15:00	-0.00541	0.00268	-1.00000	-1.00000	0.002602	0.00000	0.00000	0.126358	-0.013338	-0.104442	0.00000
max	2022-09-30 19:00:00	0.003268	0.028047	1.00000	1.00000	0.050365	1.00000	0.026586	0.218015	0.004676	0.040084	1.00000
std	NaN	0.000692	0.006192	0.577591	0.577810	0.006197	0.288905	0.001659	0.055235	0.018624	0.067938	0.362069

5. CONCLUSION

In this notebook, we have outlined and implemented the Walk Forward method using custom backtest statistics for strategies based on Deep Learning models.

With this, we finalize our journey in Deep Learning for Finance. Congrats!

We hope that all the methods described in the course help you develop your skills as an expert in Quantitative Finance. The field of Deep Learning is constantly evolving, so do not forget to keep track of any new developments!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.